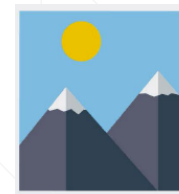




了解JavaFX基础编程模型

北京理工大学计算机学院
金旭亮

两种JavaFX界面编程模式



1

完全使用代码生成UI界面。

2

UI界面使用fxml文档以声明式的方式定义，交互逻辑及功能使用代码实现，JavaFX推荐使用MVC架构。

本讲示例采用第一种编程模式编写。

一个最简单的JavaFX程序



使用一个标签控件在窗体上显示一条信息。

示例:HelloJavaFX.java



//JavaFX程序派生自Application

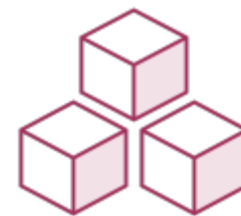
no usages

```
public class HelloJavaFXApp extends Application {  
    //程序运行时, JavaFX框架会创建一个Stage对象, 注入到start()方法中  
    public void start(Stage stage) {  
        //实例化一个JavaFX标签控件  
        Label message = new Label( text: "Hello, JavaFX!");  
        //设定内容与边界之间的距离  
        message.setPadding(new Insets( topRightBottomLeft: 10));  
        //设定文字对齐方式  
        message.setAlignment(Pos.CENTER);  
        //设定字体大小  
        message.setFont(new Font( size: 30));  
        //创建一个场景, 其中包容上面创建的标签控件对象  
        Scene scene = new Scene(message, width: 300, height: 150);  
        //将创建好的场景对象设置为“舞台 (即窗体)”的当前场景  
        stage.setScene(scene);  
        //指定窗体标题  
        stage.setTitle("学习JavaFX编程模型");  
        //显示窗体  
        stage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

```
import javafx.application.Application;  
import javafx.geometry.Insets;  
import javafx.geometry.Pos;  
import javafx.stage.Stage;  
import javafx.scene.Scene;  
import javafx.scene.control.Label;  
import javafx.scene.text.Font;
```

在IntelliJ中导入相应的包时, 注意要导入“javafx”打头的包, 而不是“awt”或“swing”的包。JavaFX与Swing/AWT的许多控件是同名的。

把握JavaFX编程模型



JavaFX应用程序派生自Application类，它提供一个launch方法启动程序：

```
//JavaFX程序派生自Application
no usages
public class HelloJavaFXApp extends Application {
    //程序运行时，JavaFX框架会创建一个Stage对象，注入到start()方法中
    > public void start(Stage stage) {...}

    public static void main(String[] args) {
        launch(args);
    }
}
```

当JavaFX程序启动时，会实例化一个HelloJavaFXApp对象，并自动调用其start()方法，此方法接收的参数是一个Stage对象（代表窗体），这个对象也由JavaFX框架负责创建。

两个JavaFX核心组件：Stage与Scene

Stage 代表“窗体”，它是一个舞台。

Scene是一个“场景”，可以在舞台上“上演”。

Scene由相应的UI控件所构建，是用户所能看到的真实内容。

传给Stage不同的Scene对象，就会导致JavaFX窗体显示不同的内容。



UI控件树

Leaf

leaf

branch

leaf

leaf

root

Scene

Stage

1

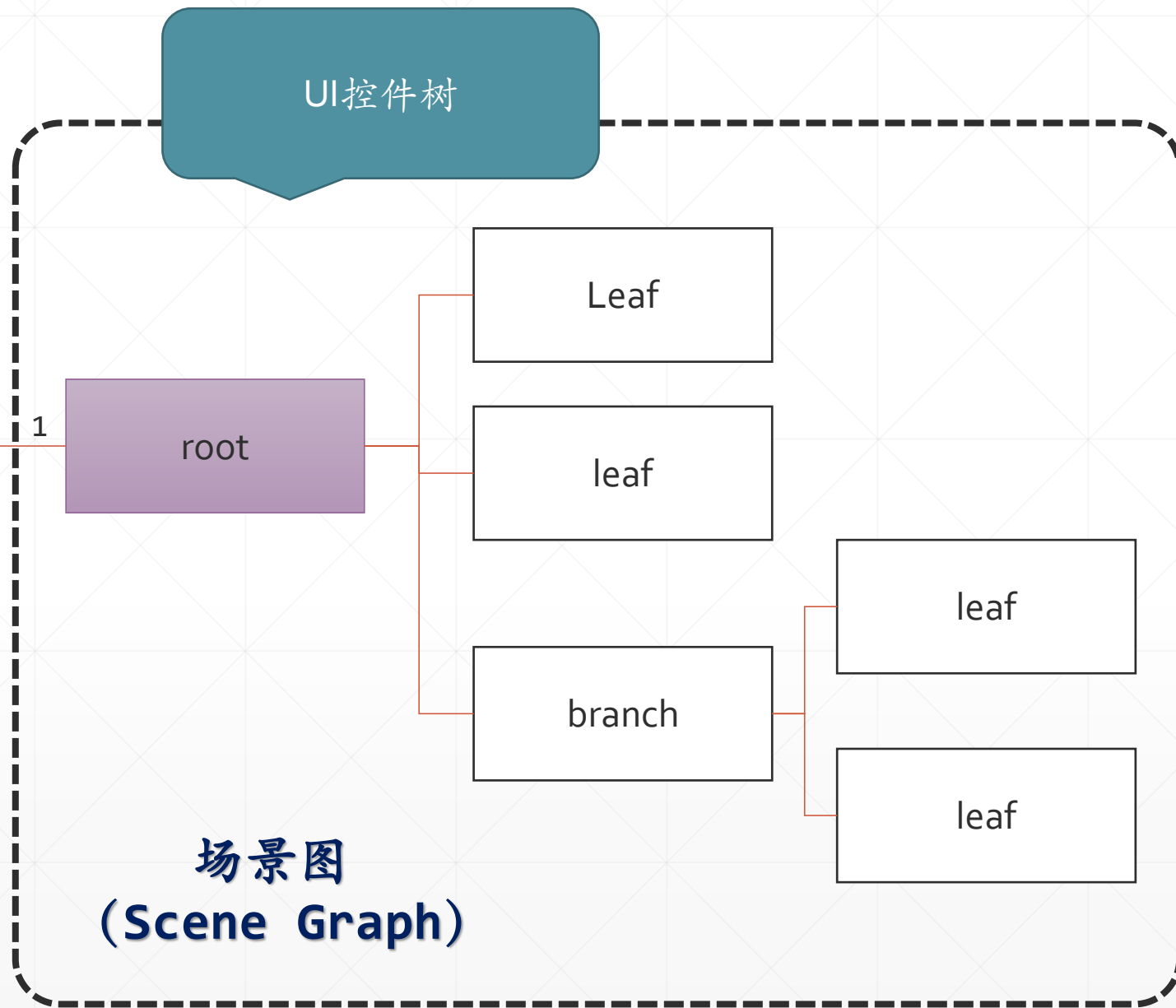
1

1

1

Stage对应于操作系统的本地窗口，具有特定操作系统特有的外观。

场景图
(Scene Graph)



小结：JavaFX编程模型



JavaFX应用程序的主类需要继承自javafx.application.**Application**类，然后重写其start()方法，此方法是所有JavaFX应用程序的入口点。



JavaFX应用程序将UI容器定义为**舞台 (Stage)** 与 **场景 (Scene)**。Stage类是顶级容器，它对应于窗体，其内容由Scene决定。Scene类是所有可视化内容的**容器 (container)**，可以把它看成是一张画布，上面绘制了用户可以看到的内容。



JavaFX应用程序的可视化界面通常由**控件 (Control)** 和 **形状 (Shape)** 构成，放到Scene中，这些对象构成一棵多叉树，称为“**Scene Graph**”。

多学一点：HostService类

在JavaFX应用中，可以使用这个类获取应用所在的文件夹相关的信息：



```
1 usage
protected Map<String, String> getHostDetails() {
    Map<String, String> map = new HashMap<>();
    HostServices host = this.getHostServices();
    //获取jar文件所在的文件夹
    String codeBase = host.getCodeBase();
    if("").equals(codeBase)){
        map.put("CodeBase", "此应用不是以jar包方式运行的");
    }else{
        map.put("CodeBase", codeBase);
    }
    //返回当前文件夹
    String documentBase = host.getDocumentBase();
    map.put("DocumentBase", documentBase);
    //获取图片资源所在的文件夹
    String splashImageURI =
        host.resolveURI(documentBase, rel: "image.jpg");
    map.put("Image URI", splashImageURI);
    return map;
}
```

打开浏览器

```
String baiduURL = "https://baidu.com";  
Button openURLButton = new Button(text: "访问百度");  
openURLButton.setOnAction(e ->  
    getHostServices().showDocument(baiduURL));
```

使用showDocument方法，会打开浏览器访问指定网址。

